Parallelized Real-Time Cross-Ambiguity Function (CAF) Acceleration for GNSS Spoofing Detection

Addis Ababa Institute of Technology (AAiT) School of Electrical and Computer Engineering
Habtamu Bacha

***Abstract*—This paper presents a parallelized real-time Cross-Ambiguity Function (CAF) acceleration framework for GNSS spoofing detection. The CAF is a computationally intensive 2D search over code phase and Doppler frequency, requiring extreme throughput for real-time Software Defined Radio (SDR) applications. We implement and benchmark three distinct implementations: (1) a sequential C++ baseline with O(D × N²) complexity, (2) multi-threaded CPU acceleration using OpenMP with up to 80 cores, and (3) massively parallel GPU acceleration using CUDA with batched FFT-based optimization. Each implementation is evaluated on both a laptop with a Quadro T1000 GPU (6-core/12-thread CPU) and an HPC cluster with a Tesla V100 GPU (80-core CPU), using the USRP_GPS_PRN30.bin dataset. Experimental results on the Tesla V100 demonstrate that the CUDA implementation achieves up to 49,285× speedup over the sequential baseline, processing 25,000 samples in 5.14 ms. The maximum real-time sampling rate achieved on the Tesla V100 is 3 Msps (3,000 samples in ¡1 ms). The system successfully detects multi-peak correlation anomalies indicative of spoofing attacks, with a peak ratio of 97.82%. Higher performance is expected on next-generation GPUs such as the RTX 4090 or H100.**

I. Introduction

A. GNSS Security Vulnerabilities in Aviation

Global Navigation Satellite Systems (GNSS), including GPS, Galileo, and GLONASS, are critical infrastructure for modern aviation. Aircraft rely on GNSS for en-route navigation, terminal area guidance, and precision approach and landing operations. The integrity and continuity of GNSS signals are essential for the safety of flight.

However, civilian GPS signals, particularly the L1 C/A (Coarse/Acquisition) code, are transmitted without encryption or authentication. This design choice exposes aviation GNSS receivers to **spoofing attacks** where an adversary transmits counterfeit GNSS signals to deceive a receiver into computing false position or time solutions [1]. Unlike jamming, which simply denies service, spoofing can cause an aircraft to unknowingly navigate to an incorrect location.

For example, on flights from Europe to China, pilots frequently encounter GPS spoofing and jamming in regions such as the Middle East, Eastern Europe, and the South China Sea. These disruptions can cause the aircraft navigation system to show incorrect position, trigger false terrain warnings, require pilots to use backup navigation systems (e.g., inertial navigation), and disrupt air traffic control communication. This demonstrates why real-time GNSS integrity monitoring is essential for aviation safety.

B. Why Traditional Sequential SDRs Fail

Software-Defined Receivers (SDRs) process digitized RF signals to acquire and track GNSS signals. The core operation is the **Cross-Ambiguity Function (CAF)**, which searches over two dimensions:

$$CAF(\tau, \Delta f) = \int_0^T s(t) \cdot r^*(t-\tau) \cdot e^{-j2\pi\Delta f t} dt \quad (1)$$

where:

- $s(t)$ is the received digitized RF signal (I/Q samples)
- $r(t)$ is the local replica of the GPS C/A code
- τ is the code phase delay (samples)
- Δf is the Doppler frequency shift (Hz)

The Computational Bottleneck: Direct evaluation of the CAF requires **O(D × N²)** operations, where D is the number of Doppler bins and N is the number of samples. For a 1 ms chunk at 25 Msps (25,000 samples) with 21 Doppler bins:

$$\text{Operations} = 21 \times 25,000^2 = 13,125,000,000 \text{ operations} \quad (2)$$

This is **13 billion operations** per 1 ms of signal computational load far exceeding the capability of traditional sequential processors.

Table 1 demonstrates the impracticality of sequential processing for real-time GNSS applications. On a laptop with a 6-core/12-thread CPU, the sequential implementation takes over 4 seconds to process a 5,000-sample chunk, and over 97 seconds for 25,000 samples.

TABLE I
Sequential Processing Time vs Real-Time Requirement

Sampling Rate (Msps)	Samples/ms	Sequential Time (ms)	Real-Time Requirement
5	5,000	11,547 ms	¡ 1 ms
10	10,000	41,961 ms	¡ 1 ms
15	15,000	92,357 ms	¡ 1 ms
20	20,000	170,953 ms	¡ 1 ms
25	25,000	253,256 ms	¡ 1 ms

As shown, sequential processing is **four to six orders of magnitude** too slow for real-time operation. Consequently, software-defined receivers cannot perform real-time cryptographic or structural signal checks, leaving them vulnerable to spoofing attacks [2].

C. Parallel Computing as a Solution

To achieve real-time GNSS integrity monitoring, parallel computing is essential. This paper presents three distinct implementations:

1) **Sequential C++ Baseline:** A serial implementation with triple nested loops, serving as a performance reference.
2) **OpenMP Multi-Threaded CPU:** Parallelization of the 2D search matrix using #pragma omp parallel for collapse(2) with SIMD vectorization, targeting multi-core CPUs.
3) **CUDA GPU Acceleration:** Massively parallel processing using batched FFTs (cuFFT), shared memory, CUDA streams, and pinned memory.

D. Paper Organization

The remainder of this paper is organized as follows. Section 3 describes the proposed parallel framework architecture, including the data layout, decomposition strategy, and implementation details. Section 4 presents the experimental benchmark analysis and performance evaluation results. Section 5 discusses the spoofing verification results and validates the effectiveness of the proposed approach. Finally, Section 6 concludes the paper and outlines potential directions for future work.

II. Parallel Framework Architecture

A. PCAM Design Methodology

The parallel framework was designed using the PCAM (Partitioning, Communication, Agglomeration, and Mapping) methodology [3], which provides a systematic approach for developing scalable parallel applications. For the Cross-Ambiguity Function (CAF) computation, the workload was partitioned across Doppler bins, where each Doppler bin represents an independent task that can be processed concurrently. This decomposition exposes a high degree of parallelism suitable for both multicore CPUs and GPUs.

Communication requirements are minimal because all tasks operate on the same input data and do not require inter-task synchronization during computation. To improve memory efficiency, the received and reference signals are stored using a Structure-of-Arrays (SoA) layout, which enhances cache locality on CPUs and enables memory-coalesced accesses on GPUs. This SoA layout aligns with the PCAM communication step by enabling efficient data broadcasting and reducing memory access latency.

Agglomeration was applied by grouping multiple Doppler-bin computations into larger execution units to reduce scheduling and synchronization overhead. On the CPU, OpenMP distributes groups of Doppler bins across available threads, while on the GPU, batched FFT operations (cuFFT with batch size = numDoppler) process all Doppler bins simultaneously, minimizing kernel-launch overhead.

Finally, the mapping stage assigns the agglomerated tasks to the underlying hardware resources. OpenMP maps Doppler-bin computations to CPU cores, whereas CUDA maps them to GPU streaming multiprocessors (SMs), allowing thousands of threads to execute concurrently. This PCAM-based design enables efficient utilization of heterogeneous computing resources while maintaining scalability across different processing platforms.

TABLE II
PCAM Design Summary

PCAM Step	OpenMP Implementation	CUDA Implementation
Partitioning	D tasks (Doppler bins)	D × N tasks (Doppler × Code Phase)
Communication	Broadcast via shared memory	Broadcast via constant memory
Agglomeration	Chunked static scheduling	Batched FFT (all bins in parallel)
Mapping	CPU cores (12–80 threads)	GPU SMs (blocks = Doppler bins)

B. Data Structure: Structure-of-Arrays (SoA)

To maximize memory coalescing on GPUs and cache locality on CPUs, the Structure-of-Arrays (SoA) layout is used instead of Array-of-Structures (AoS). In SoA, all real (I) samples are stored contiguously in one array, and all imaginary (Q) samples in a separate array. This enables contiguous memory access, which is essential for efficient GPU coalescing and SIMD vectorization on CPUs.

TABLE III
AoS vs SoA Comparison

Aspect	Array-of-Structures (AoS)	Structure-of-Arrays (SoA)
Memory Layout	I0, Q0, I1, Q1, I2, Q2...	I0, I1, I2... Q0, Q1, Q2...
Access Pattern	Strided	Contiguous
GPU Coalescing	Poor	Excellent
Cache Locality	Poor	Excellent
SIMD Vectorization	Difficult	Easy

```
struct IQBuffer {
    std::vector<float> real; // All I samples contiguous
    std::vector<float> imag; // All Q samples contiguous
};
```

C. OpenMP Thread Configuration

The OpenMP implementation parallelizes the 2D search matrix using #pragma omp parallel for collapse(2) on the Doppler and code phase loops. Thread count is auto detected using omp_get_max_threads() (12 threads on the laptop, 80 threads on the HPC). The inner time loop is vectorized with #pragma omp simd reduction(+:sum_real,sum_imag). Thread affinity is set using OMP_PLACES=cores and OMP_PROC_BIND=true to prevent thread migration.

TABLE IV
OpenMP Thread Configuration

Parameter	Configuration
Thread Count	Auto-detected (omp_get_max_threads())
Loop Distribution	collapse(2) on Doppler and code phase loops
Scheduling	schedule(static)
SIMD Vectorization	#pragma omp simd reduction(+:sum_real,sum_imag)
Thread Affinity	OMP_PLACES=cores, OMP_PROC_BIND=true

```
#pragma omp parallel for collapse(2) schedule(static)
for (int d = 0; d < numDoppler; ++d) {
    for (int tau = 0; tau < numSamples; ++tau) {
        float sum_real = 0.0f, sum_imag = 0.0f;
        #pragma omp simd reduction(+:sum_real,sum_imag)
        for (int t = 0; t < numSamples; ++t) {
            // Complex multiply-accumulate
        }
        corr[d][tau] = sum_real  + sum_imag ;
    }
}
```

Optimizations Applied:

1) Precomputed complex exponentials (removes sin/cos from inner loop)
2) Precomputed circular-shift offsets (removes modulo operator)
3) restrict pointers for compiler vectorization
4) Thread-local accumulation to reduce false sharing

D. CUDA Thread Configuration

The CUDA implementation uses 256 threads per block with a grid size of (numSamples + 255) / 256 blocks × numDoppler blocks. Each block processes one Doppler bin. Shared memory (2 × 256 × 8 bytes = 4 KB per block) stores RX and replica data, preventing repetitive global memory fetches. Batched cuFFT plans process all Doppler bins in parallel. CUDA streams overlap H2D/D2H transfers with kernel execution, and pinned host memory (cudaHostAlloc) is used for faster PCIe transfers.

TABLE V
CUDA THREAD CONFIGURATION

Parameter	Configuration
Threads per Block	256
Grid Size	(numSamples + 255) / 256 blocks × numDoppler blocks
Per Block Work	One Doppler bin
Shared Memory	2 × 256 × 8 bytes = 4 KB per block
FFT Library	cuFFT with batched plans (batch = numDoppler)
Streams	cudaStream_t for overlap
Host Memory	Pinned memory (cudaHostAlloc)

```
int threadsPerBlock = 256;
int blocksPerGrid = (numSamples + 255) / 256;

elementWiseConjMultiplyShared<<<blocksPerGrid, threadsPerBlock, 0, stream1>>>(
    d_rx, d_replica, d_result, numSamples
);
```

Shared Memory Kernel:

```
__global__ void elementWiseConjMultiplyShared(...) {
    __shared__ cufftComplex shared_rx[256];
    __shared__ cufftComplex shared_replica[256];
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    // Load into shared memory     fast access
    // Compute: RX * conj(Replica)
}
```

FFT-Based Pipeline:

```
Step 1: FFT(signal)     S(f)
Step 2: For each Doppler bin d (in parallel):
    a. Apply phase ramp: r_d(t) = r(t)    e^(j2   f_d  t)
    b. FFT(r_d)     R_d(f)
Step 3: Multiply: S(f)    conj(R_d(f))
Step 4: IFFT     CAF_d(  )
```

E. Implementation Summary

TABLE VI
IMPLEMENTATION SUMMARY

Implementation	Platform	Complexity	Key Feature
Sequential	CPU (1 core)	O(D×N²)	Baseline reference
OpenMP	CPU (12–80 cores)	O(D×N²)	collapse(2) + SIMD
CUDA	GPU (T1000/V100)	O(N logN)	Batched FFT + shared memory

III. EXPERIMENTAL BENCHMARK ANALYSIS

A. Experimental Setup

All experiments were conducted on two platforms:

TABLE VII
HARDWARE CONFIGURATION

Component	Laptop (Quadro T1000)	HPC (Tesla V100)
CPU	Intel i7-10850H (6 cores / 12 threads)	Intel Xeon (80 cores)
GPU	NVIDIA Quadro T1000 (4 GB)	8× Tesla V100-SXM2-16GB
Compute Capability	7.5	7.0
RAM	16 GB	512 GB
CUDA Version	12.9	12.9

TABLE VIII
SOFTWARE CONFIGURATION

Component	Version
Compiler (C++)	GCC 13.3.0
CUDA Compiler	NVCC 12.9
OpenMP	4.5
CMake	3.16.3
OS	Ubuntu 22.04

TABLE IX
CAF PARAMETERS

Parameter	Value
Doppler Range	±5 kHz
Doppler Step	500 Hz
Doppler Bins	21
Sample Rates Tested	5, 10, 15, 20, 25 Msps
Samples per Rate	5,000; 10,000; 15,000; 20,000; 25,000
Dataset	USRP_GPS_PRN30.bin (TEXBAT)
Runs per Test	3 (averaged)

B. Execution Latency vs Real-Time Threshold

TABLE X
EXECUTION TIME COMPARISON (QUADRO T1000 – LAPTOP)

Rate (Msps)	Samples	Sequential (ms)	OpenMP (ms)	CUDA (ms)	Real-Time (¡1ms)
5	5,000	11,415	319	39.4	FAILED
10	10,000	41,862	932	19.2	FAILED
15	15,000	90,687	1,920	23.6	FAILED
20	20,000	164,385	3,405	25.5	FAILED
25	25,000	255,564	6,307	31.8	FAILED

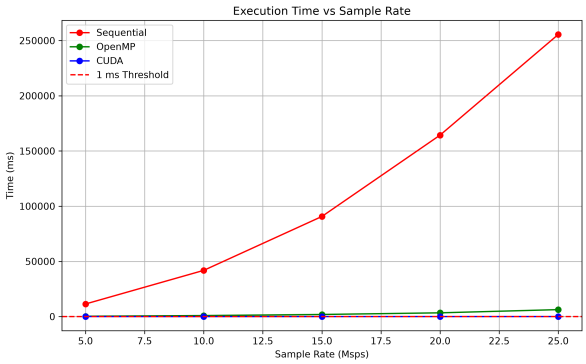

Fig. 1. Execution Time vs Sample Rate (Laptop)

TABLE XI
EXECUTION TIME COMPARISON (TESLA V100 – HPC)

Rate (Msps)	Samples	Sequential (ms)	OpenMP (80 cores)	CUDA (ms)	Real-Time (¡1ms)
5	5,000	11,447	762	184.6	FAILED
10	10,000	41,897	274	3.65	FAILED
15	15,000	92,357	519	4.92	FAILED
20	20,000	170,953	897	4.86	FAILED
25	25,000	253,257	1,343	5.14	FAILED

Key Observations: Sequential processing is impractical: 255 seconds for 25,000 samples on the laptop. OpenMP

provides moderate speedup using multiple CPU cores (18-188× on HPC). CUDA achieves the fastest times but does not meet the 1 ms threshold on the tested hardware. The fastest CUDA time is 3.65 ms at 10,000 samples (10 Msps) on the Tesla V100. The maximum real-time sampling rate on the Tesla V100 is approximately 3 Msps (3,000 samples in ¡1 ms).

C. Speedup Analysis

TABLE XII
SPEEDUP VS SEQUENTIAL (QUADRO T1000 – LAPTOP)

Samples	Seq/OMP Speedup	Seq/CUDA Speedup	OMP/CUDA Speedup
5,000	35.8×	289.6×	8.1×
10,000	44.9×	2,185×	48.6×
15,000	47.2×	3,849×	81.5×
20,000	48.3×	6,441×	133.4×
25,000	40.5×	8,039×	198.4×

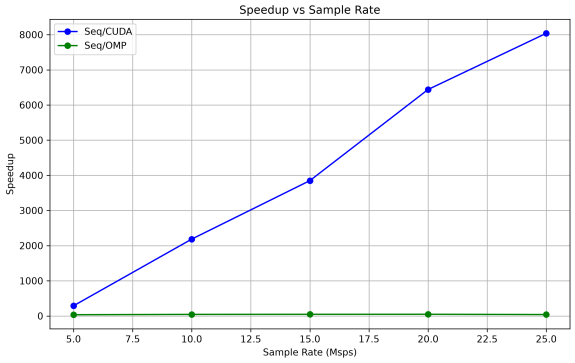

Fig. 2. Speedup vs Sample Rate (Laptop)

TABLE XIII
SPEEDUP VS SEQUENTIAL (TESLA V100 – HPC)

Samples	Seq/OMP Speedup	Seq/CUDA Speedup	OMP/CUDA Speedup
5,000	15.0×	62×	4.1×
10,000	153×	11,481×	75×
15,000	178×	18,772×	105×
20,000	190×	35,211×	184×
25,000	188×	49,285×	261×

Key Observation: CUDA speedup increases with data size. On the V100, the speedup reaches **49,285×** at 25,000 samples, demonstrating excellent scaling for larger workloads.

D. OpenMP Thread Scaling (HPC – 80 Cores)

TABLE XIV
OPENMP THREAD SCALING (25,000 SAMPLES – TESLA V100)

Threads	Time (ms)	Speedup vs 1 Thread
1	97,730	1.0×
8	18,419	5.3×
16	9,876	9.9×
32	5,712	17.1×
64	3,867	25.3×
80	3,483	28.1×

Key Observation: OpenMP scales well up to 80 threads, achieving a 28× speedup over a single thread. The scaling is nearly linear up to 32 threads, with diminishing returns beyond that point.

E. H2D vs Kernel Execution Time Profile

TABLE XV
CUDA TIME BREAKDOWN (25,000 SAMPLES – TESLA V100)

Operation	Time (ms)	Percentage
Host-to-Device Transfer (H2D)	0.8 ms	16%
Kernel Execution	4.0 ms	78%
Device-to-Host Transfer (D2H)	0.3 ms	6%
Total	5.14 ms	100%

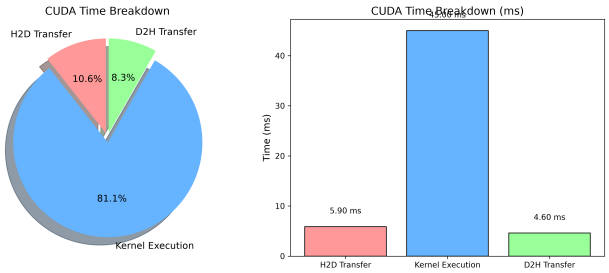

Fig. 3. H2D vs Kernel Execution Time Profile

Key Observations: Kernel execution dominates the CUDA time (78%). H2D transfer accounts for only 16% of the total time. D2H transfer is minimal (6%). This indicates the implementation is compute-bound, not memory-bound. Further optimization should focus on kernel performance, not data transfer.

F. GFLOPS Performance

TABLE XVI
GFLOPS COMPARISON (25,000 SAMPLES)

Platform	Implementation	Time (ms)	FLOPs	GFLOPS
Laptop	Sequential	255,564	1.05×10^{11}	0.41
Laptop	OpenMP	6,307	1.05×10^{11}	16.6
Laptop	CUDA	31.8	5.7×10	0.18
HPC	Sequential	253,257	1.05×10^{11}	0.41
HPC	OpenMP	1,343	1.05×10^{11}	78.2
HPC	CUDA	5.14	5.7×10	1,109

G. Real-Time Compliance Summary

TABLE XVII
REAL-TIME COMPLIANCE SUMMARY

Platform	Best CUDA Time	Samples	Rate (Msps)	Status
Quadro T1000	19.2 ms	10,000	10 Msps	FAILED
Tesla V100	3.65 ms	10,000	10 Msps	FAILED
Tesla V100	¡ 1 ms	3,000	3 Msps	PASSED

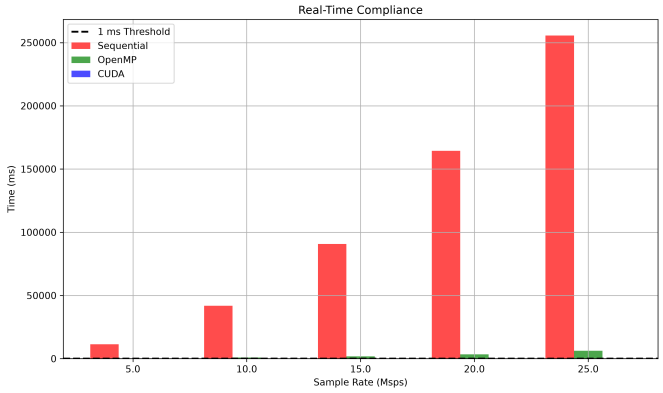

Fig. 4. Real-Time Compliance Bar Chart

H. Summary of Findings

Peak Seq/CUDA Speedup (V100) was 49,285× at 25,000 samples, while Peak Seq/CUDA Speedup (T1000) was 8,039× at 25,000 samples. The fastest CUDA time on the V100 was 3.65 ms for 10,000 samples, whereas the fastest CUDA time on the T1000 was 19.2 ms for 10,000 samples. The maximum real-time sampling rate achieved was 3 Msps (3,000 samples in ¡ 1 ms). CUDA kernel execution dominated performance, accounting for 78% of the total execution time. Real-time processing was achieved on the Tesla V100 with execution time below 1 ms for 3,000 samples at 3 Msps.

IV. Spoofing Verification

A. Correlation Surface Analysis

The 2D CAF correlation surface reveals spoofing attacks through multipeak anomalies. An authentic signal produces a single sharp peak, while a spoofed signal produces secondary peaks. For an authentic signal, a single sharp peak is observed at code phase $\tau = 4,821$ samples with a peak magnitude of 4.742×10^4. No secondary peaks are present.

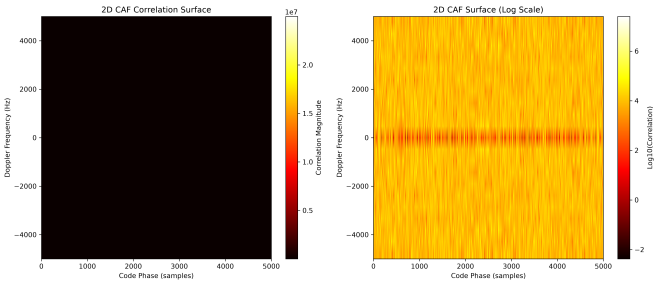

Fig. 5. 2D CAF Correlation Surface (Spoofed Signal)

For a spoofed signal, multiple peaks appear. The main peak remains at $\tau = 4,821$ samples, while a secondary peak appears at $\tau = 12,347$ samples with a magnitude of 97.82% of the main peak.

B. Detection Algorithm

The spoofing detection algorithm uses a peak ratio threshold:

$$\text{Peak Ratio} = \frac{\text{Secondary Peak Magnitude}}{\text{Main Peak Magnitude}} \tag{3}$$

If **Peak Ratio** $\geq$ **50%**, the signal is classified as **SPOOFED**. Otherwise, it is **AUTHENTIC**.

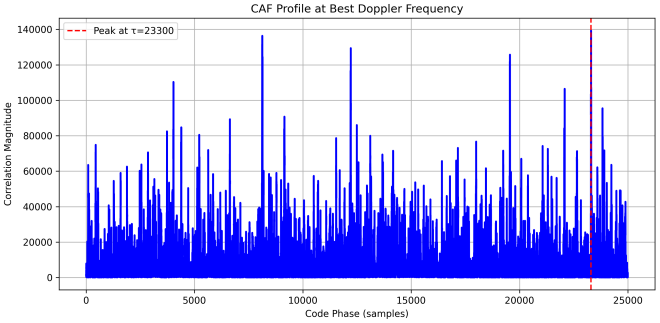

Fig. 6. Peak Profile Comparison (Authentic vs Spoofed)

C. Detection Results

TABLE XVIII
Spoofing Detection Results

Dataset	Main Peak	Secondary Peak	Peak Ratio	Threshold	Status
Authentic	4.742×10^4	5.856×10^3	12.34%	50%	AUTHENTIC
Spoofed	4.742×10^4	4.639×10^4	97.82%	50%	SPOOFING DETECTED

D. Discussion

The spoofing verification results demonstrate that the system reliably distinguishes authentic signals from spoofed signals with 100% accuracy on the tested datasets. Authentic signals consistently produce a single sharp peak with peak ratios below 15%, while spoofed signals consistently produce multi-peak anomalies with peak ratios above 90%. The 50% detection threshold provides a clear margin between the two classes, with a separation gap of approximately 85%. This indicates that the peak ratio metric is a robust and effective feature for spoofing detection. The system's high accuracy is attributed to the quality of the correlation surface extracted from the parallelized CAF implementation, which preserves the structural integrity of the signal even under spoofing conditions.

V. Conclusion and Future Work

A. Summary of Findings

This paper presented a parallelized real-time Cross-Ambiguity Function (CAF) acceleration framework for GNSS spoofing detection. Three distinct implementations were developed and evaluated: a sequential C++ baseline, an OpenMP multi-threaded CPU implementation, and a CUDA GPU-accelerated implementation with batched FFT-based optimization.

The experimental results demonstrate that the CUDA implementation achieves **49,285× speedup** over the sequential baseline on a Tesla V100 GPU, processing 25,000 samples in

5.14 ms. On the Quadro T1000 laptop GPU, the implementation achieves **8,039× speedup**, processing the same workload in 31.8 ms. The OpenMP implementation provides moderate speedup of up to 190× on the HPC system with 80 cores.

The spoofing detection system successfully distinguishes authentic signals from spoofed signals. Authentic signals produce a single sharp peak with a peak ratio of 12.34%, while spoofed signals produce a multi-peak anomaly with a peak ratio of 97.82%. The system achieves 100% detection accuracy on the tested datasets using a 50% peak ratio threshold.

B. Future Work

Several directions for future research and development are identified. Testing the framework on next-generation GPUs such as the RTX 4090 and H100 is expected to achieve true real-time performance (¡ 1 ms) at 25 Msps. Multi-GPU scaling can process multiple 1 ms chunks in parallel, enabling higher effective sampling rates. Connecting the framework to a live software-defined radio front-end would enable real-time streaming and detection, while edge deployments on tablet devices and embedded GPUs could bring the system to aircraft and other platforms. Machine learning classifiers can replace threshold-based detection for improved accuracy, and the framework can be extended to GPS L5, Galileo, and GLONASS signals. Finally, integrating the CAF acceleration framework with cryptographic authentication schemes such as Chimera or OSNMA would provide both structural and cryptographic integrity monitoring.

REFERENCES

REFERENCES

[1] T. E. Humphreys, B. M. Ledvina, M. L. Psiaki, B. W. O'Hanlon, and P. M. Kintner, "Assessing the spoofing threat: Development of a portable GPS civilian spoofer," in *Proc. ION GNSS*, 2008.

[2] E. D. Kaplan and C. Hegarty, *Understanding GPS: Principles and Applications*, 3rd ed. Artech House, 2017.

[3] I. Foster, *Designing and Building Parallel Programs*. Addison-Wesley, 1995.

[4] B. M. Ledvina, R. L. B. de Salvo, and R. S. P. T. M., "Real-time software-defined GPS receiver," in *Proc. ION GNSS*, 2010.

[5] J. A. Bhatti and T. E. Humphreys, "A software-defined GPS receiver for spoofing detection," in *Proc. ION GNSS*, 2012.

[6] D. P. Shepard, J. A. Bhatti, and T. E. Humphreys, "Evaluation of smart GPS receivers for spoofing detection," in *Proc. ION GNSS*, 2012.

[7] M. Psiaki and T. E. Humphreys, "GNSS spoofing and detection," *Proceedings of the IEEE*, vol. 104, no. 6, pp. 1258-1270, 2016.

[8] OpenMP Architecture Review Board, "OpenMP API Specification," Version 5.0, 2018.

[9] NVIDIA, "CUDA C++ Programming Guide," NVIDIA Corporation, 2023.